

CICLO FORMATIVO DE GRADO SUPERIOR

DESARROLLO DE APLICACIONES MULTIPLATAFORMA (DAM)

TRABAJO FIN DE GRADO

**Desarrollo de una aplicación web corporativa para el
emprendimiento artesanal Makisá**

*Aplicación web con arquitectura cliente-servidor desarrollada en HTML5, CSS3 y
JavaScript, con backend en Node.js y Express, y persistencia de datos en MySQL*

Autora:

Alejandra Varona Gaviria

Centro educativo:

Colegio Sagrada Familia

Curso académico: 2025/2026

Fecha de entrega: 25 de junio de 2026

Zaragoza, España

ÍNDICE

ÍNDICE	2
1. Introducción	3
1.1 Descripción del proyecto	3
1.2 Objetivos	3
1.3 Alcance	4
1.4 Justificación del proyecto	4
2. Análisis y diseño de la identidad de marca	5
2.1 Construcción de la identidad visual	5
3. Tecnologías utilizadas	7
3.1 HTML5	7
3.2 CSS3	7
3.3 JavaScript	7
3.4 Node.js y Express	8
3.5 MySQL	9
3.6 Git y GitHub	10
3.7 Netlify	11
4. Diseño e implementación	13
4.1 Arquitectura de la aplicación	13
4.2 Diseño de la interfaz	13
4.3 Diseño responsive	14
4.4 Organización del catálogo de productos	16
4.5 Sistema de filtrado del catálogo	18
4.6 Diseño de la página de producto	20
4.7 Formulario de contacto	21
4.8 Desarrollo del backend	22
4.9 Base de datos	24
4.10 Control de versiones	25
4.11 Despliegue de la aplicación	25
5. Manual de usuario	27
5.1 Acceso a la aplicación	27
5.2 Navegación por el sitio web	27
5.3 Página principal	27
5.4 Consulta del catálogo de productos	28
5.5 Uso del sistema de filtros	29
5.6 Consulta del detalle de un producto	30
5.7 Sección "Sobre mí"	30
5.8 Página de contacto	31
5.9 Diseño responsive	31
6. Desarrollo técnico	33
6.1 Entorno de desarrollo	33
6.2 Herramientas y tecnologías	33
6.3 Estructura del proyecto	33
6.4 Instalación del proyecto	35

6.4.1 Clonado del repositorio	35
6.4.2 Instalación de dependencias	35
6.5 Configuración de la base de datos	36
6.6 Configuración del servidor	36
6.7 Puesta en marcha del servidor	36
6.8 API REST implementada	36
6.9 Despliegue de la aplicación	37
6.10 Mantenimiento y ampliaciones futuras	37
7. Pruebas y validación	39
7.1 Problemas encontrados y soluciones adoptadas	39
8. Uso de herramientas de Inteligencia Artificial	40
8.1 Diseño de la identidad visual	40
8.2 Implementación del sistema de filtrado	40
8.3 Carga dinámica de productos	40
8.4 Corrección de errores CSS	40
8.5 Implementación del formulario y backend	41
9. Conclusiones	42
9.1 Líneas de trabajo futuras	42
10. Bibliografía y referencias	43
Referencias generales	43
Diseño responsive y experiencia de usuario	44
APIs y desarrollo web	44
Recursos gráficos	44

1. Introducción

1.1 Descripción del proyecto

Makisá es un emprendimiento artesanal dedicado a la creación de accesorios hechos a mano e ilustraciones botánicas inspiradas en la naturaleza. Actualmente, la presencia digital de muchos pequeños emprendimientos depende principalmente de redes sociales, lo que limita la organización de la información y dificulta la presentación estructurada de productos y servicios.

Con el objetivo de ofrecer una plataforma propia que represente la identidad visual de la marca, se ha desarrollado una aplicación web que permite mostrar el catálogo de productos, consultar información sobre el proyecto y facilitar la comunicación con potenciales clientes mediante un formulario de contacto.

El proyecto se ha desarrollado siguiendo una arquitectura cliente-servidor sencilla, utilizando tecnologías ampliamente empleadas en el desarrollo web moderno. La aplicación combina tecnologías de frontend para la interfaz de usuario y tecnologías backend para la gestión de datos y la comunicación con la base de datos.

Además de los conocimientos adquiridos durante el ciclo formativo, se han incorporado herramientas adicionales como Node.js, Express y Netlify, permitiendo ampliar la experiencia práctica en el desarrollo de aplicaciones web completas.

1.2 Objetivos

El objetivo principal del proyecto es diseñar e implementar una aplicación web funcional para la marca Makisá.

Para alcanzar este objetivo general se han definido los siguientes objetivos específicos:

- Diseñar una interfaz visual atractiva y coherente con la identidad de la marca.
- Desarrollar una navegación sencilla e intuitiva.
- Mostrar un catálogo organizado por categorías.
- Permitir visualizar información detallada de cada producto.
- Implementar un sistema de contacto mediante formulario web.
- Almacenar las consultas recibidas en una base de datos MySQL.
- Aplicar buenas prácticas de desarrollo utilizando control de versiones mediante Git y GitHub.
- Publicar la aplicación en Internet mediante Netlify.

1.3 Alcance

El alcance del proyecto se centra en la creación de una plataforma web corporativa orientada a la presentación de productos artesanales y a la recepción de consultas por parte de los usuarios.

No se contempla la implementación de funcionalidades avanzadas como comercio electrónico, pasarela de pagos, gestión de usuarios o administración interna de productos, ya que dichas características exceden los objetivos establecidos para este trabajo.

1.4 Justificación del proyecto

La justificación de este proyecto reside en la necesidad real de pequeños emprendimientos artesanales de disponer de una presencia digital propia que no dependa exclusivamente de plataformas de redes sociales. Una página web corporativa aporta mayor credibilidad, organización de contenidos y capacidad de personalización de la imagen de marca.

Desde el punto de vista formativo, el proyecto permite poner en práctica de manera integrada las competencias adquiridas durante el Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma, incorporando además el aprendizaje autónomo de tecnologías backend como Node.js y Express.

2. Análisis y diseño de la identidad de marca

2.1 Construcción de la identidad visual

Uno de los aspectos más importantes del proyecto ha sido trasladar la identidad de la marca Makisá al entorno digital.

Inicialmente se planteó el desarrollo de la aplicación utilizando la paleta de colores existente en el logotipo del emprendimiento, basada principalmente en tonos azules, magentas y verdes. Sin embargo, tras las primeras pruebas de diseño se observó que la combinación generaba una apariencia excesivamente institucional, similar a la de páginas corporativas o gubernamentales, alejándose de la imagen cercana y artesanal que se pretendía transmitir.

Por este motivo se decidió realizar un rediseño completo de la identidad visual de la aplicación. La nueva propuesta se inspiró en elementos relacionados con la naturaleza, la ilustración botánica, los materiales artesanales y los colores presentes en flores y frutos.



Figura 1. Propuesta inicial de identidad visual de la aplicación Makisá con paleta azul-magenta-verde.

La paleta definitiva se compone de los siguientes colores:

Tabla 1. Paleta de colores definitiva de la identidad visual de Makisá.

Color	Código HEX	Uso principal
Vino	#7A004A	Color principal y elementos destacados
Magenta	#D80A8C	Botones y elementos interactivos
Salmón	#F28A7A	Gradientes y elementos secundarios

Color	Código HEX	Uso principal
Melocotón	#F0C06E	Elementos decorativos
Crema	#FAF7F2	Fondo principal
Arena	#F1ECE3	Fondos alternativos

La utilización de colores cálidos permite transmitir cercanía, creatividad y un carácter artesanal, reforzando la personalidad de la marca.

Asimismo, se seleccionaron las tipografías Cormorant Garamond para títulos y encabezados, y Poppins para textos y contenidos informativos. La combinación de ambas fuentes proporciona un equilibrio entre elegancia y legibilidad.

Finalmente, se incorporaron ilustraciones botánicas como elementos decorativos en distintas secciones de la aplicación con el objetivo de reforzar la temática natural del proyecto y aportar una mayor personalidad visual al sitio web.



Figura 2. Identidad visual definitiva de Makisá con paleta de colores cálidos e ilustraciones botánicas.

3. Tecnologías utilizadas

3.1 HTML5

HTML5 se ha utilizado como lenguaje de marcas para estructurar el contenido de todas las páginas que forman la aplicación. Gracias a HTML5 se han definido los distintos elementos que componen la interfaz, como menús de navegación, formularios, secciones informativas, tarjetas de productos y pies de página. La estructura semántica implementada facilita la organización del contenido y mejora la accesibilidad de la aplicación.

3.2 CSS3

CSS3 se ha empleado para diseñar la apariencia visual de la aplicación. Se han utilizado hojas de estilo personalizadas para definir:

- Colores corporativos.
- Tipografías.
- Distribución de elementos.
- Adaptación a distintos tamaños de pantalla.
- Efectos visuales y transiciones.

El diseño sigue una línea estética inspirada en la naturaleza y la ilustración botánica, manteniendo coherencia con la identidad de la marca Makisá.

3.3 JavaScript

JavaScript constituye uno de los componentes más importantes del proyecto, ya que permite añadir interactividad y comportamiento dinámico a la aplicación. Su utilización evita recargas innecesarias de páginas y mejora la experiencia del usuario. Entre las funcionalidades implementadas destacan:

- Filtrado dinámico de productos.
- Lectura de parámetros desde la URL.
- Carga automática de información de productos.
- Gestión del formulario de contacto.
- Comunicación asíncrona con el servidor mediante Fetch API.

Por ejemplo, el sistema de filtrado permite mostrar únicamente los productos pertenecientes a una categoría determinada sin necesidad de recargar la página.

Asimismo, JavaScript permite obtener el identificador de un producto desde la URL y cargar automáticamente su información correspondiente. Gracias a este mecanismo se evita la creación de una página HTML independiente para cada producto, facilitando la escalabilidad del catálogo.

3.4 Node.js y Express

Para el desarrollo del backend se ha utilizado Node.js junto con el framework Express. Node.js permite ejecutar código JavaScript fuera del navegador, posibilitando la creación de aplicaciones servidor. Express simplifica la creación de rutas HTTP y la gestión de peticiones y respuestas.

En este proyecto se ha desarrollado una API sencilla encargada de recibir los datos enviados desde el formulario de contacto y almacenarlos en la base de datos. La API expone un endpoint de tipo POST encargado de recibir los datos enviados desde el formulario de contacto, validarlos y almacenarlos de forma persistente en la base de datos. La conexión con MySQL se realiza mediante la librería `mysql2`, que permite ejecutar consultas SQL desde el servidor Node.js.

```

40 app.post("/contacto", (req, res) => {
41
42     console.log("PETICION RECIBIDA");
43     console.log(req.body);
44
45     // Obtiene los datos enviados por el cliente
46     const { nombre, email, mensaje } = req.body;
47
48     /*
49     * Consulta SQL parametrizada para insertar
50     * un nuevo contacto en la base de datos
51     */
52     const sql = `
53         INSERT INTO contactos
54         (nombre, email, mensaje)
55         VALUES (?, ?, ?)
56     `;
57
58     conexion.query(
59         sql,
60         [nombre, email, mensaje],
61         (error, resultado) => {
62
63             // Gestión de errores durante la inserción
64             if (error) {
65                 console.error(error);
66
67                 return res.status(500).json({
68                     error: "Error al guardar"
69                 });
70             }
71
72             // Respuesta enviada al cliente si el registro se guarda correctamente
73             res.json({
74                 mensaje: "Consulta guardada correctamente"
75             });
76         }
77     )

```

Figura 3. Esquema de la arquitectura Node.js y Express utilizada en el backend de la aplicación.

3.5 MySQL

MySQL se ha utilizado como sistema gestor de bases de datos relacional. Su función principal es almacenar las consultas realizadas por los usuarios a través del formulario de contacto. Cada mensaje queda registrado con los siguientes datos:

- Nombre del usuario.
- Correo electrónico.
- Mensaje o consulta.
- Fecha y hora de envío.

La utilización de MySQL permite disponer de almacenamiento persistente y facilita futuras ampliaciones del proyecto.

```
12  /*
13  |  * Configuración de la conexión con la base de datos MySQL
14  |  */
15  const conexion = mysql.createConnection({
16  |      host: "localhost",
17  |      user: "root",
18  |      password: "1234",
19  |      database: "makisa"
20  |  });
```

Figura 4. Estructura de la base de datos MySQL de la aplicación Makisá.

3.6 Git y GitHub

Durante el desarrollo de la aplicación se ha utilizado Git como sistema de control de versiones. Esta herramienta permite registrar de forma cronológica todos los cambios realizados en el código fuente, facilitando el seguimiento de la evolución del proyecto y permitiendo trabajar de una forma más organizada y segura.

El uso de Git ha aportado diversas ventajas durante el desarrollo:

- Control de versiones: cada modificación realizada en el proyecto queda registrada mediante commits, lo que permite conocer qué cambios se han realizado, cuándo se realizaron y con qué finalidad.
- Recuperación de cambios: en caso de errores o modificaciones no deseadas, Git permite volver a versiones anteriores del proyecto sin perder el trabajo realizado previamente.
- Copia de seguridad remota: el proyecto se ha almacenado en un repositorio remoto mediante GitHub, evitando la pérdida de información y permitiendo disponer de una copia actualizada del código desde cualquier equipo con acceso a Internet.
- Desarrollo incremental: la aplicación se ha construido de forma progresiva, implementando nuevas funcionalidades y realizando mejoras de manera continua.

La utilización conjunta de Git y GitHub constituye una práctica habitual en el desarrollo profesional de software, ya que mejora la organización del proyecto, incrementa la seguridad del código y facilita el mantenimiento y la futura ampliación de la aplicación.

```
1 • CREATE DATABASE makisa;
2
3 • USE makisa;
4
5 • CREATE TABLE contactos (
6     id INT AUTO_INCREMENT PRIMARY KEY,
7     nombre VARCHAR(100) NOT NULL,
8     email VARCHAR(100) NOT NULL,
9     mensaje TEXT NOT NULL,
10    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );
```

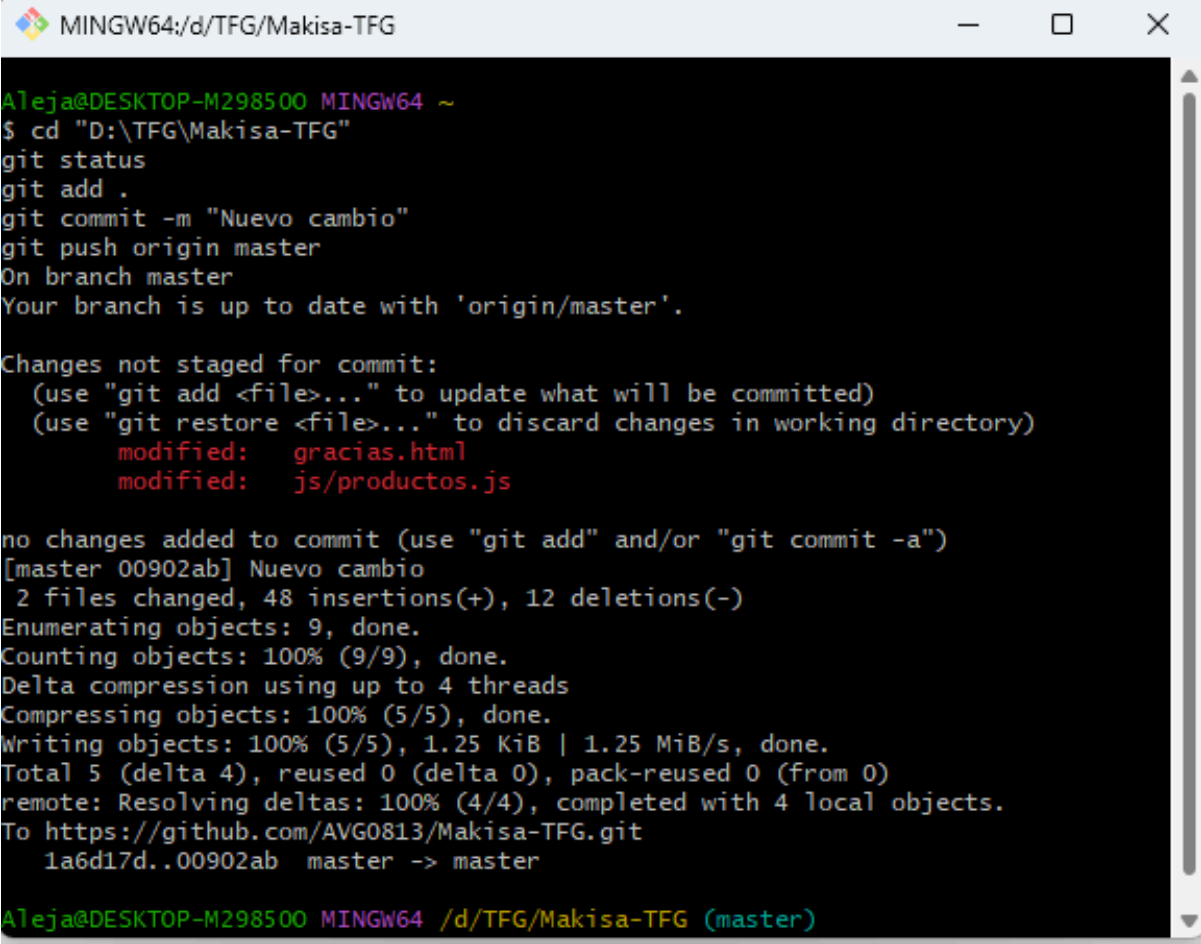
Figura 5. Historial de commits del repositorio GitHub del proyecto Makisá.

3.7 Netlify

La publicación de la aplicación web se ha realizado mediante Netlify. Esta plataforma permite desplegar aplicaciones estáticas directamente desde un repositorio GitHub. Cada vez que se realiza un nuevo push al repositorio, la plataforma genera automáticamente una nueva versión de la aplicación y la publica en Internet. Gracias a ello, la aplicación puede ser accesible desde cualquier dispositivo con conexión a Internet sin necesidad de realizar configuraciones adicionales en servidores propios.

El flujo de despliegue implementado es el siguiente:

- Repositorio GitHub → Netlify → <https://makisa.netlify.app>



```
MINGW64:/d/TFG/Makisa-TFG
Aleja@DESKTOP-M298500 MINGW64 ~
$ cd "D:\TFG\Makisa-TFG"
git status
git add .
git commit -m "Nuevo cambio"
git push origin master
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   gracias.html
        modified:   js/productos.js

no changes added to commit (use "git add" and/or "git commit -a")
[master 00902ab] Nuevo cambio
 2 files changed, 48 insertions(+), 12 deletions(-)
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.25 KiB | 1.25 MiB/s, done.
Total 5 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/AVG0813/Makisa-TFG.git
 1a6d17d..00902ab master -> master

Aleja@DESKTOP-M298500 MINGW64 /d/TFG/Makisa-TFG (master)
```

Figura 6. Panel de control de Netlify mostrando el despliegue automático de la aplicación Makisá.

4. Diseño e implementación

4.1 Arquitectura de la aplicación

La aplicación sigue una arquitectura cliente-servidor sencilla. El frontend, desarrollado mediante HTML, CSS y JavaScript, se ejecuta en el navegador del usuario y se encarga de la presentación de la información y de la interacción con la interfaz. El backend, desarrollado con Node.js y Express, se encarga de procesar las peticiones recibidas y gestionar el almacenamiento de datos en MySQL. La comunicación entre ambos componentes se realiza mediante peticiones HTTP utilizando formato JSON.

La arquitectura implementada puede representarse de la siguiente manera:

- Usuario → Frontend (HTML + CSS + JavaScript) → API REST (Node.js + Express) → Base de datos MySQL

master	1 Branch	0 Tags	Go to file	Add file	Code
AVG0813	Nuevo cambio	00902ab · 1 minute ago	10 Commits		
CSS	version def	1 hour ago			
img	version def	1 hour ago			
js	Nuevo cambio	1 minute ago			
node_modules	version def	1 hour ago			
TFG.docx	version def	1 hour ago			
catalogo.html	version def	1 hour ago			
contacto.html	version def	1 hour ago			
gracias.html	Nuevo cambio	1 minute ago			
index.html	version def	1 hour ago			
makisa.sql	version def	1 hour ago			
mensajes.json	version def	1 hour ago			
package-lock.json	version def	1 hour ago			
package.json	version def	1 hour ago			

Figura 7. Diagrama de arquitectura cliente-servidor de la aplicación Makisá.

4.2 Diseño de la interfaz

Uno de los principales objetivos del proyecto ha sido crear una aplicación visualmente atractiva y coherente con la identidad de la marca Makisá. La estética de la página está inspirada en la naturaleza y la ilustración botánica, utilizando una paleta de colores cálidos y orgánicos compuesta por tonos vino, magenta, salmón y melocotón.

Para mantener una identidad visual uniforme se definieron variables CSS globales mediante el selector `:root`, permitiendo reutilizar los colores corporativos en toda la aplicación y facilitando futuras modificaciones.

El diseño se estructura en secciones claramente diferenciadas:

- Página principal.
- Catálogo de productos.
- Página de detalle de producto.
- Página "Sobre mí".
- Página de contacto.

La navegación se ha diseñado para ser sencilla e intuitiva, permitiendo al usuario acceder rápidamente a cualquier sección de la aplicación.

```
1  /* Variables de color utilizadas en todo el sitio */
2  :root{
3      --vino: #7A004A;
4      --magenta: #D80A8C;
5      --salmon: #F28A7A;
6      --melocoton: #F0C06E;
7
8      --crema: #FAF7F2;
9      --arena: #F1ECE3;
10
11     --texto: #3B2F2F;
12     --blanco: #FFFFFF;
13 }
```

Figura 8. Variables CSS globales definidas para la identidad visual de Makisá.

4.3 Diseño responsive

La aplicación ha sido desarrollada siguiendo un enfoque responsive para garantizar una correcta visualización en diferentes dispositivos. Para ello se han utilizado Flexbox, CSS Grid y Media Queries. El contenido se adapta automáticamente a distintos tamaños de pantalla, reorganizando los elementos cuando el ancho del dispositivo es reducido.

Ejemplos de adaptación responsive:

- El menú de navegación pasa a mostrarse en varias líneas en pantallas pequeñas.

- Las tarjetas de productos se reorganizan en una única columna en dispositivos móviles.
- Las secciones con dos columnas se convierten en una distribución vertical.

Gracias a ello la aplicación puede utilizarse tanto en ordenadores de escritorio como en tablets y dispositivos móviles.

```
34      <!-- CABECERA: navegación principal del sitio -->
35      <header>
36          <nav>
37              <!-- Logo textual del proyecto -->
38              <h1 class="logo">Makisá</h1>
39
40              <!-- Menú de navegación estático entre páginas -->
41              <ul>
42                  <li><a href="index.html">Inicio</a></li>
43                  <li><a href="catalogo.html">Catálogo</a></li>
44                  <li><a href="sobre-mi.html">Sobre mí</a></li>
45                  <li><a href="contacto.html">Contacto</a></li>
46              </ul>
47          </nav>
48      </header>
```

Figura 9. Vista de la aplicación Makisá en dispositivos de escritorio.


```

872  /* RESPONSIVE */
873
874  @media (max-width:768px){
875
876      nav{
877          flex-direction:column;
878          gap:15px;
879      }
880
881      nav ul{
882          flex-wrap:wrap;
883          justify-content:center;
884      }
885
886      .hero h1{
887          font-size:3rem;
888      }
889
890      .hero h2{
891          font-size:2rem;
892      }
893
894      .contacto{
895          grid-template-columns:1fr;
896      }
897
898      .producto-detalle{
899          grid-template-columns:1fr;
900      }
901
902      .pagina-titulo h1{
903          font-size:3rem;
904      }
905      .catalogo-decoracion{
906          display:none;
907      }
908  }
909  @media (max-width:900px){
910
911      .producto-detalle{
912          grid-template-columns:1fr;
913          gap:40px;
914          text-align:center;
915      }
916
917      #producto-nombre{
918          font-size:2.2rem;

```

Figura 10. Vista de la aplicación Makisá en dispositivo móvil mostrando el diseño responsive.

4.4 Organización del catálogo de productos

La organización del catálogo de productos se diseñó inicialmente como una arquitectura basada en páginas HTML independientes para cada producto. Sin embargo, este enfoque generaba una alta duplicación de archivos y dificultaba el mantenimiento y escalabilidad del proyecto.

Por este motivo se adoptó una solución dinámica basada en JavaScript, en la que toda la información de los productos se centraliza en una estructura de datos en el lado cliente. Los productos se almacenan en un objeto JavaScript que actúa como una base de datos local, donde cada clave representa el identificador único del producto. Cada producto contiene los siguientes atributos:

- Nombre.

- Categoría.
- Precio.
- Imagen.
- Descripción.

Cuando el usuario selecciona un producto desde el catálogo, se envía un identificador mediante parámetros en la URL (por ejemplo: producto.html?id=flor-fucsia). Posteriormente, JavaScript interpreta este parámetro, busca el producto correspondiente dentro del objeto y renderiza dinámicamente la información en la vista de detalle.

Esta implementación aporta varias ventajas:

- Reducción de duplicación de código HTML.
- Mayor facilidad de mantenimiento.
- Escalabilidad sencilla para añadir nuevos productos.
- Separación entre estructura visual y datos.

```
1  const productos = {
2
3    "flor-fucsia": {
4      nombre: "Flor Fucsia",
5      categoria: "Aretes",
6      precio: "$35.000 COP",
7      imagen: "img/productos/arete-flor-fucsia-1.jpeg",
8      descripcion:
9        "Aretes artesanales inspirados en formas botánicas y elaborados cuidadosamente a mano."
10   },
11
12   "topo-rosa-blanco": {
13     nombre: "Topo Rosa y Blanco",
14     categoria: "Aretes",
15     precio: "$20.000 COP",
16     imagen: "img/productos/topo-rosa-blanco.jpeg",
17     descripcion:
18       "Topos artesanales inspirados en la naturaleza."
19   },
20
21   "topo-azul-negro": {
22     nombre: "Topo Azul y Negro",
23     categoria: "Aretes",
24     precio: "$20.000 COP",
25     imagen: "img/productos/topo-azul-negro-1.jpeg",
26     descripcion:
27       "Pieza artesanal hecha a mano."
28   },
29 }
```

Figura 11. Objeto JavaScript con la estructura de datos del catálogo de productos (base de datos frontend).

```

121 // Obtener los parámetros de la URL
122 const params = new URLSearchParams(window.location.search);
123
124 // Recuperar el identificador del producto
125 const id = params.get("id");
126
127 // Buscar el producto correspondiente
128 const producto = productos[id];
129
130 // Si el producto existe, cargar la información
131 if (producto) {
132     document.getElementById("producto-imagen").src =
133         producto.imagen;
134     document.getElementById("producto-nombre").textContent =
135         producto.nombre;
136     document.getElementById("producto-categoria").textContent =
137         producto.categoria;
138     document.getElementById("producto-precio").textContent =
139         producto.precio;
140     document.getElementById("producto-descripcion").textContent =
141         producto.descripcion;
142 }
143
144
145
146
147

```

Figura 12. Función JavaScript encargada de la carga dinámica de productos por parámetro de URL.

4.5 Sistema de filtrado del catálogo

Con el objetivo de mejorar la experiencia de usuario, se implementó un sistema de filtrado dinámico en el catálogo de productos mediante JavaScript. Este sistema permite mostrar únicamente los productos pertenecientes a una categoría seleccionada sin necesidad de recargar la página, mejorando así la fluidez de navegación.

Cada tarjeta de producto almacena su categoría mediante el atributo personalizado `data-categoria`, mientras que los botones de filtrado utilizan el atributo `data-filtro`. El funcionamiento del sistema es el siguiente:

- El usuario selecciona un botón de filtro.
- Se elimina la clase visual del filtro activo anterior.
- Se asigna la clase al nuevo filtro seleccionado.
- Se obtiene la categoría asociada al botón.

- Se recorren todos los productos del catálogo.
- Se muestran únicamente los productos cuya categoría coincide con el filtro seleccionado.

Adicionalmente, el sistema permite aplicar un filtro automáticamente mediante parámetros en la URL, lo que facilita el acceso directo a categorías específicas (por ejemplo: `catalogo.html?categoria=stickers`).

```

1 // Botones de filtro del catálogo
2 const botones = document.querySelectorAll(".filtros button");
3
4 // Productos del catálogo
5 const productos = document.querySelectorAll(".productos-catalogo .card-link");
6

```

Figura 13. Selección de elementos del DOM para el sistema de filtrado del catálogo.

```

7 // Evento para cada botón
8 botones.forEach(boton => {
9
10     boton.addEventListener("click", () => {
11
12         // Quitar filtro activo anterior
13         const activo = document.querySelector(".filtro-activo");
14         if (activo) activo.classList.remove("filtro-activo");
15
16         // Activar botón actual
17         boton.classList.add("filtro-activo");
18
19         // Filtro seleccionado
20         const filtro = boton.dataset.filtro.toLowerCase();
21
22         // Filtrar productos
23         productos.forEach(producto => {
24
25             const card = producto.querySelector(".card");
26             const categoria = card ? card.dataset.categoria.toLowerCase() : "";
27
28             if (filtro === "todos" || categoria === filtro) {
29                 producto.style.display = ""; // deja que CSS decida
30             } else {
31                 producto.style.display = "none";
32             }
33         });
34     });
35 });
36
37

```

Figura 14. Lógica JavaScript del sistema de filtrado dinámico del catálogo.

```

39 // Filtro por URL (?categoria=aretes)
40 const parametros = new URLSearchParams(window.location.search);
41 const categoriaURL = parametros.get("categoria");
42
43 if (categoriaURL) {
44     const boton = document.querySelector(`[data-filtro="${categoriaURL}"]`);
45     if (boton) boton.click();
46 }

```

Figura 15. Implementación del filtro por parámetro de URL en el catálogo de productos.

4.6 Diseño de la página de producto

La página de detalle del producto se diseñó con el objetivo de destacar las imágenes y la información de cada pieza artesanal. Uno de los principales problemas encontrados durante el desarrollo fue la gran diferencia de tamaños y proporciones entre las fotografías de los productos.

Para solucionarlo se implementó una estructura común basada en:

- Propiedad aspect-ratio para definir una proporción uniforme.
- Propiedad object-fit: contain para adaptar la imagen sin deformarla.
- Un contenedor con dimensiones fijas.

Gracias a esta solución todas las imágenes mantienen un tamaño uniforme sin deformarse, consiguiendo una presentación visual más consistente.

```

827 #producto-imagen{
828     width:100%;
829     max-width:500px;
830     aspect-ratio:1/1;
831
832     margin:auto;
833
834     object-fit:contain;
835
836     background: #F7F4EF;
837     border-radius:25px;
838     padding:30px;
839
840     box-shadow:0 10px 30px rgba(0,0,0,.08);
841 }

```

Figura 16. Vista de la página de detalle de producto de la aplicación Makisá.

4.7 Formulario de contacto

El formulario de contacto constituye el principal mecanismo de comunicación entre los usuarios y la marca Makisá, permitiendo que cualquier persona interesada en los productos pueda realizar consultas, solicitar información adicional o plantear encargos personalizados.

Inicialmente se estudió la posibilidad de utilizar el sistema de formularios proporcionado por Netlify. Sin embargo, durante las pruebas realizadas se detectaron diversas limitaciones relacionadas con la configuración y la gestión de los datos enviados, por lo que finalmente se optó por implementar una solución propia basada en Node.js y Express.

El formulario desarrollado recoge tres datos fundamentales:

- Nombre del usuario.
- Dirección de correo electrónico.
- Mensaje o consulta.

Desde el punto de vista de la interfaz, el formulario se implementó utilizando elementos HTML5, aprovechando los atributos `required` y `type="email"` para realizar una primera validación en el navegador antes de enviar la información al servidor.

El flujo completo de funcionamiento puede representarse de la siguiente manera:

- Usuario → Formulario HTML → Petición HTTP POST → Servidor Node.js + Express → Base de datos MySQL → Respuesta al usuario

La utilización de consultas parametrizadas aporta una mayor seguridad a la aplicación, ya que evita la concatenación directa de cadenas SQL y reduce el riesgo de ataques de inyección SQL.

La implementación de este formulario ha permitido integrar en un mismo desarrollo diferentes tecnologías: HTML5 para la creación de la interfaz, JavaScript para la gestión del envío de datos, Node.js y Express para el procesamiento de peticiones HTTP y MySQL para el almacenamiento persistente de la información.

```

89      <form class="formulario" id="contactoForm">
90
91      <!-- Campo anti-spam -->
92      <p hidden>
93      |   <label>
94      |       No rellenes esto:
95      |   <input name="bot-field">
96      |   </label>
97      </p>
98
99      <!-- Datos introducidos por el usuario -->
100     <input
101     |   type="text"
102     |   name="nombre"
103     |   placeholder="Tu nombre"
104     |   required>
105
106     <input
107     |   type="email"
108     |   name="email"
109     |   placeholder="Tu correo electrónico"
110     |   required>
111
112     <textarea
113     |   name="mensaje"
114     |   rows="6"
115     |   placeholder="Cuéntame en qué puedo ayudarte..."
116     |   required>
117     </textarea>
118
119     <!-- Botón de envío -->
120     <button type="submit">
121     |   Enviar mensaje
122     </button>
123
124     </form>

```

Figura 17. Formulario de contacto de la aplicación Makisá con campos de validación HTML5.

Contacto

¿Tienes alguna consulta?

Hablemos

Si tienes alguna pregunta, quieres realizar un pedido personalizado o colaborar con Makisá, estaré encantada de escucharte.

Email:
isabellavar@unicauca.edu.co

WhatsApp:
+57 310 822 2286

Ubicación:
Colombia

Enviar mensaje

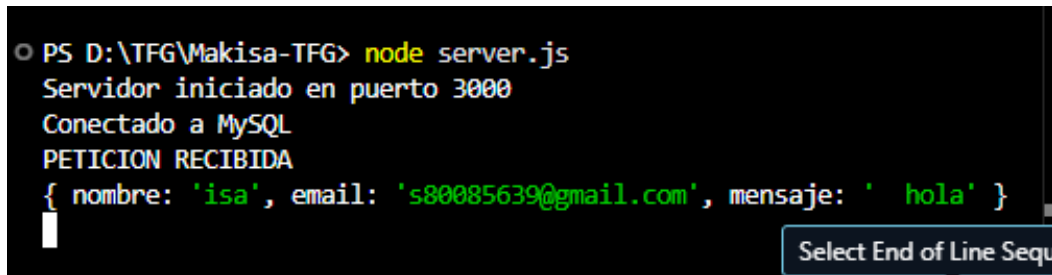
Figura 18. Código JavaScript para el envío asíncrono de datos del formulario de contacto mediante Fetch API.

4.8 Desarrollo del backend

Se decidió ampliar el alcance del proyecto incorporando nuevas herramientas y conocimientos relacionados con el desarrollo del lado del servidor. Para ello se utilizaron Node.js, Express y MySQL. Se desarrolló una pequeña API REST encargada de:

- Recibir las peticiones del formulario.
- Validar los datos recibidos.
- Almacenar la información en la base de datos.
- Enviar una respuesta al cliente.

El servidor se implementó en el archivo `server.js`, convirtiéndose en uno de los componentes más importantes del proyecto. La utilización de un backend propio ha permitido adquirir conocimientos adicionales, especialmente relacionados con APIs REST, peticiones HTTP, middleware, comunicación cliente-servidor y persistencia de datos.



```
PS D:\TFG\Makisa-TFG> node server.js
Servidor iniciado en puerto 3000
Conectado a MySQL
PETICION RECIBIDA
{ nombre: 'isa', email: 's80085639@gmail.com', mensaje: ' hola' }
```

Figura 19. Extracto del código del servidor Node.js con la configuración de Express y la conexión a MySQL.


```

37  * Endpoint que recibe los datos enviados desde
38  * el formulario de contacto del sitio web
39  */
40  app.post("/contacto", (req, res) => {
41
42      console.log("PETICION RECIBIDA");
43      console.log(req.body);
44
45      // Obtiene los datos enviados por el cliente
46      const { nombre, email, mensaje } = req.body;
47
48      /*
49       * Consulta SQL parametrizada para insertar
50       * un nuevo contacto en la base de datos
51       */
52      const sql = `
53          INSERT INTO contactos
54          (nombre, email, mensaje)
55          VALUES (?, ?, ?)
56      `;
57
58      conexion.query(
59          sql,
60          [nombre, email, mensaje],
61          (error, resultado) => {
62
63              // Gestión de errores durante la inserción
64              if (error) {
65                  console.error(error);
66
67                  return res.status(500).json({
68                      error: "Error al guardar"
69                  });
70              }
71
72              // Respuesta enviada al cliente si el registro se guarda correctamente
73              res.json({
74                  mensaje: "Consulta guardada correctamente"
75              });

```

Figura 20. Implementación del endpoint POST /contacto en el servidor Node.js.

4.9 Base de datos

La aplicación utiliza una base de datos MySQL denominada makisa. La tabla principal del proyecto es contactos, encargada de almacenar las consultas enviadas desde el formulario.

Tabla 2. Estructura de la tabla contactos de la base de datos Makisá.

Campo	Tipo	Descripción
id	INT AUTO_INCREMENT PRIMARY KEY	Identificador único del registro
nombre	VARCHAR(100) NOT NULL	Nombre del usuario que realiza la consulta
email	VARCHAR(100) NOT NULL	Correo electrónico del usuario
mensaje	TEXT NOT NULL	Contenido del mensaje enviado

Campo	Tipo	Descripción
fecha	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Fecha y hora de registro automático

Cada consulta queda almacenada de forma permanente, permitiendo futuras ampliaciones del sistema.

	id	nombre	email	mensaje	fecha
▶	1	varona gaviria alejandra	alejandravarona1308@gmail.com	asFERGHY	2026-06-23 22:02:13
	2	alejandra	alejandravarona1308@gmail.com	sdsfg	2026-06-23 22:09:19
	3	adf	alejandravarona1308@gmail.com	wrrgtyj	2026-06-23 22:15:40
	4	isa	s80085639@gmail.com	hola	2026-06-24 21:29:48
*	NULL	NULL	NULL	NULL	NULL

Figura 21. Vista de la tabla contactos en MySQL Workbench con registros de prueba.

4.10 Control de versiones

Durante el desarrollo se han realizado numerosas modificaciones y mejoras en el proyecto. Entre las más destacables se encuentran:

- Implementación del catálogo dinámico.
- Incorporación del sistema de filtros.
- Rediseño responsive.
- Ajuste de tamaños de imágenes.
- Mejora de la navegación entre páginas.
- Integración del backend y la base de datos.
- Corrección de enlaces y funcionalidades.

Todas estas modificaciones han sido gestionadas mediante Git y GitHub, permitiendo mantener un historial completo del desarrollo y recuperar versiones anteriores en caso de errores.

4.11 Despliegue de la aplicación

La versión final de la aplicación se ha publicado utilizando Netlify. El despliegue se realiza directamente desde el repositorio de GitHub, permitiendo automatizar las actualizaciones de la página. Cada vez que se realiza un nuevo commit en la rama principal, Netlify genera automáticamente una nueva versión del sitio web. Este sistema facilita enormemente la publicación y el mantenimiento de la aplicación sin necesidad de administrar un servidor propio.

5. Manual de usuario

El presente apartado describe el funcionamiento de la aplicación desde el punto de vista del usuario final, explicando las diferentes secciones del sitio web y la forma de interactuar con cada una de ellas. La aplicación ha sido diseñada con el objetivo de ofrecer una navegación sencilla e intuitiva, permitiendo que cualquier usuario pueda acceder a la información de manera rápida, independientemente de sus conocimientos técnicos.

5.1 Acceso a la aplicación

La aplicación se encuentra disponible a través de Internet mediante la siguiente dirección: <https://makisa.netlify.app>. El acceso puede realizarse desde cualquier navegador web moderno, tanto en ordenadores de escritorio como en tablets y dispositivos móviles. Al acceder a la página principal, el usuario encuentra una presentación de la marca Makisá junto con un menú de navegación situado en la parte superior de la pantalla.

5.2 Navegación por el sitio web

La aplicación dispone de un menú principal presente en todas las páginas del sitio web. Este menú permite acceder rápidamente a las diferentes secciones: Inicio, Catálogo, Sobre mí y Contacto. La navegación es completamente estática y no requiere ningún tipo de registro o autenticación.



Figura 22. Menú de navegación principal de la aplicación web Makisá.

5.3 Página principal

La página principal constituye la carta de presentación del proyecto. En ella se muestra una breve descripción de la marca, accesos directos al catálogo de productos, elementos visuales relacionados con la identidad de Makisá y botones de navegación hacia otras secciones de la aplicación. El objetivo de esta página es transmitir la filosofía de la marca y ofrecer un primer contacto con los productos artesanales.

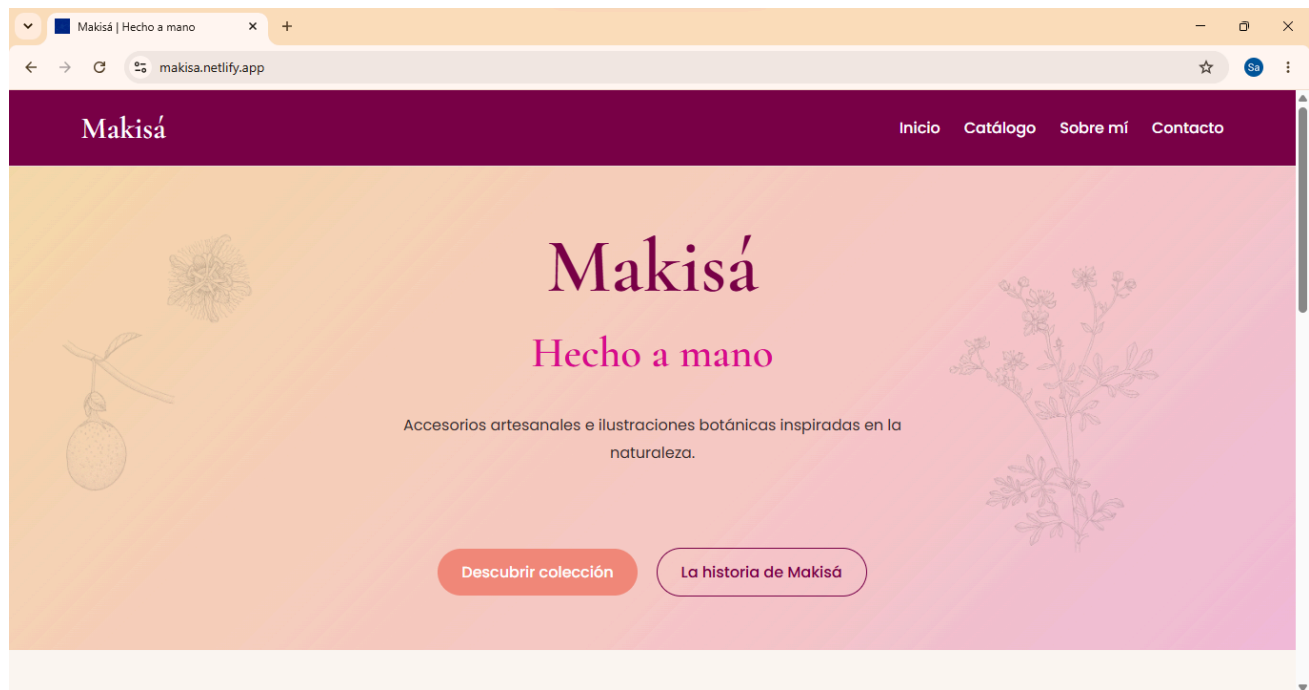


Figura 23. Página principal de la aplicación web Makisá.



Figura 24. Sección de presentación de la página principal de Makisá.

5.4 Consulta del catálogo de productos

La página de catálogo permite visualizar todos los productos disponibles organizados mediante tarjetas. Cada tarjeta muestra la imagen del producto, nombre, categoría y precio. La disposición de las tarjetas se adapta automáticamente al tamaño de la pantalla gracias al diseño responsive implementado.

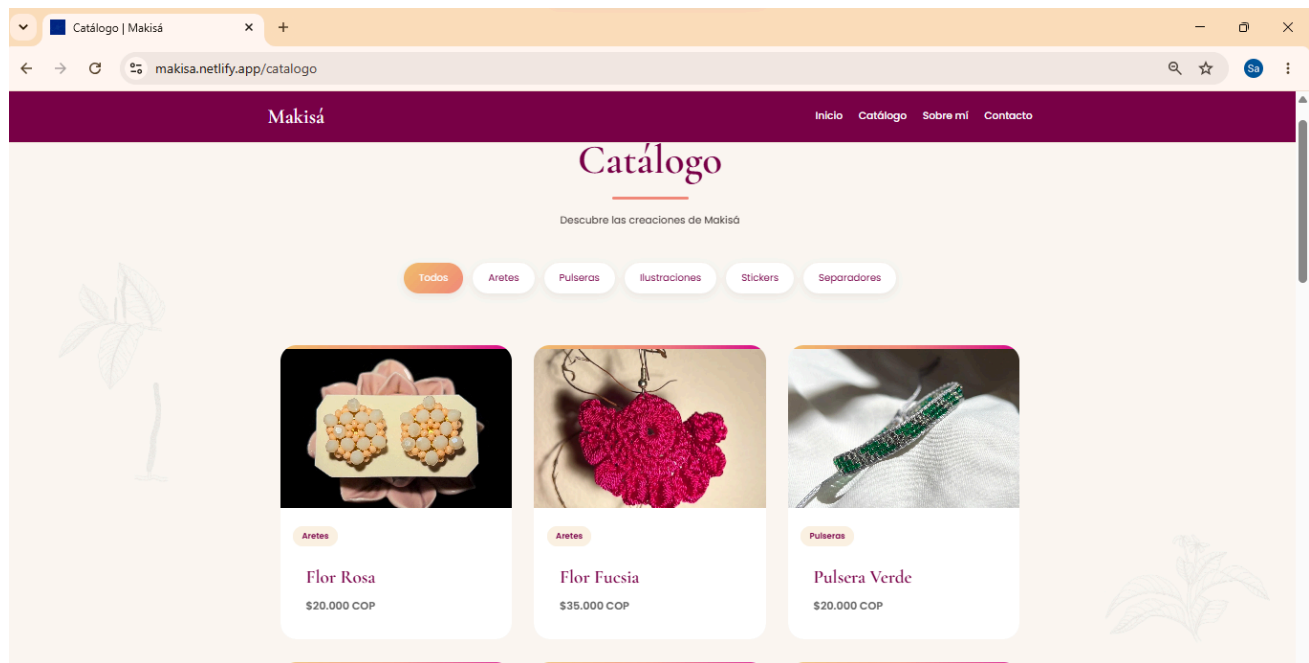


Figura 25. Catálogo de productos de la aplicación Makisá con sistema de tarjetas.

5.5 Uso del sistema de filtros

Con el objetivo de facilitar la búsqueda de productos, el catálogo incorpora un sistema de filtrado por categorías. Las categorías disponibles son: Todos, Aretes, Pulseras, Stickers y Separadores. Cuando el usuario pulsa uno de los botones de filtrado, se selecciona la categoría deseada y se muestran únicamente los productos correspondientes sin necesidad de recargar la página.



Figura 26. Sistema de filtrado por categorías del catálogo de productos de Makisá.

5.6 Consulta del detalle de un producto

Al seleccionar un producto del catálogo, el usuario es redirigido a la página de detalle correspondiente. En ella se muestra una imagen ampliada del producto, su nombre, categoría, precio y descripción detallada. Además, se incluye un botón que permite contactar con la marca para solicitar más información o realizar un encargo personalizado. Todo el proceso se realiza sin recargar la página, proporcionando una experiencia de usuario más fluida.

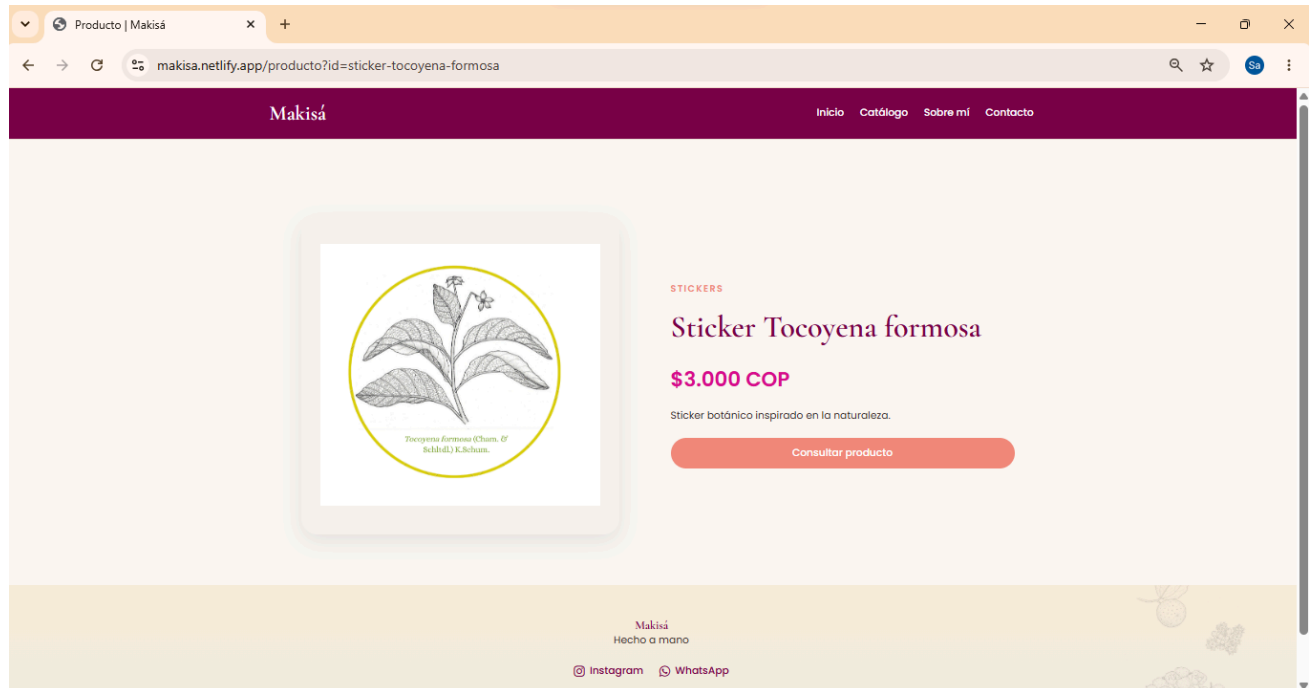


Figura 27. Página de detalle de producto de la aplicación Makisá.

5.7 Sección "Sobre mí"

La sección "Sobre mí" presenta la historia y filosofía detrás de la marca Makisá, incluyendo información sobre la artesana, sus técnicas de trabajo y la relación entre la biología, la ilustración botánica y la artesanía.



Figura 28. Sección "Sobre mí" de la aplicación web Makisá.

5.8 Página de contacto

La página de contacto permite a los usuarios comunicarse directamente con la marca Makisá. El formulario recoge el nombre, dirección de correo electrónico y mensaje del usuario. Además, se incluyen enlaces a las redes sociales de la marca, permitiendo al usuario comunicarse a través de otros medios.



Figura 29. Página de contacto de la aplicación web Makisá con formulario y enlaces a redes sociales.

5.9 Diseño responsive

La interfaz se adapta automáticamente al tamaño de la pantalla, reorganizando los elementos para ofrecer una experiencia óptima en dispositivos móviles, tablets y ordenadores de escritorio.



Figura 30. Vista de la aplicación Makisá en dispositivo móvil mostrando la adaptación responsive del catálogo.

6. Desarrollo técnico

6.1 Entorno de desarrollo

Para el desarrollo del proyecto se han utilizado las siguientes herramientas:

- Visual Studio Code: editor de código principal.
- GitHub: repositorio remoto y control de versiones.
- Netlify: despliegue y publicación del frontend.
- MySQL Workbench: administración y gestión de la base de datos.

6.2 Herramientas y tecnologías

Tabla 3. Tecnologías y herramientas utilizadas en el desarrollo del proyecto.

Tecnología	Versión	Uso en el proyecto
HTML5	-	Estructura de todas las páginas
CSS3	-	Diseño y estilos visuales
JavaScript (ES6+)	-	Lógica de frontend e interactividad
Node.js	>=18.x	Entorno de ejecución del backend
Express	^4.x	Framework para la API REST
MySQL	>=8.0	Sistema gestor de base de datos
mysql2	^3.x	Librería de conexión Node.js-MySQL
cors	^2.x	Middleware para solicitudes cross-origin
Git	-	Control de versiones
GitHub	-	Repositorio remoto
Netlify	-	Despliegue del frontend
Visual Studio Code	-	Editor de código
MySQL Workbench	-	Administración de base de datos

6.3 Estructura del proyecto

El proyecto se encuentra organizado mediante una estructura de carpetas sencilla con el objetivo de facilitar su mantenimiento:

```
makisa/  
|  
├─ index.html  
├─ catalogo.html  
├─ producto.html  
├─ sobre-mi.html  
└─ contacto.html
```

```
|
|├── CSS/
|│   └── style.css
|
|├── js/
|│   ├── catalogo.js
|│   ├── productos.js
|│   └── contacto.js
|
|├── img/
|│   ├── productos/
|│   └── ilustraciones/
|
|├── server.js
|├── package.json
|└── package-lock.json
```

Tabla 4. Descripción de los archivos principales del proyecto Makisá.

Archivo	Función
index.html	Página principal
catalogo.html	Catálogo de productos
producto.html	Página de detalle de producto
sobre-mi.html	Información sobre la marca
contacto.html	Formulario de contacto
style.css	Estilos de toda la aplicación
catalogo.js	Sistema de filtrado
productos.js	Carga dinámica de productos
contacto.js	Gestión del formulario de contacto
server.js	Backend de la aplicación

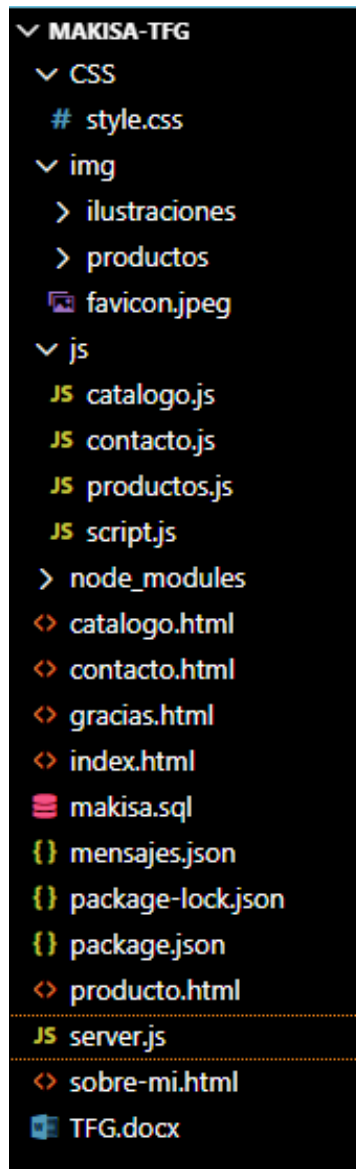


Figura 31. Estructura de carpetas del proyecto Makisá en Visual Studio Code.

6.4 Instalación del proyecto

6.4.1 Clonado del repositorio

Para obtener una copia local del proyecto es necesario ejecutar el siguiente comando:

```
git clone https://github.com/AVG0813/Makisa-TFG.git
```

6.4.2 Instalación de dependencias

Una vez descargado el proyecto se instalan las dependencias de Node.js:

```
npm install
```

Este comando instala automáticamente las librerías definidas en el archivo `package.json`. Las principales dependencias utilizadas son:

```
{  
  "express": "^4.x",  
}
```

```
"cors": "^2.x",  
"mysql2": "^3.x"  
}
```

6.5 Configuración de la base de datos

La aplicación utiliza una base de datos denominada makisa. Para crearla es necesario ejecutar:

```
CREATE DATABASE makisa;
```

Posteriormente se crea la tabla encargada de almacenar las consultas recibidas desde el formulario de contacto:

```
CREATE TABLE contactos(  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL,  
  mensaje TEXT NOT NULL,  
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

6.6 Configuración del servidor

La conexión con la base de datos se realiza desde el archivo server.js. La configuración utilizada es la siguiente:

```
const conexion = mysql.createConnection({  
  host: "localhost",  
  user: "root",  
  password: "1234",  
  database: "makisa"  
});
```

Estos parámetros deberán modificarse en función de la configuración de cada equipo.

6.7 Puesta en marcha del servidor

Una vez configurada la base de datos, el servidor puede iniciarse mediante:

```
node server.js
```

Si la conexión se realiza correctamente aparecerán los siguientes mensajes:

```
Conectado a MySQL  
Servidor iniciado en puerto 3000
```

El servidor quedará escuchando peticiones en: <http://localhost:3000>

6.8 API REST implementada

La aplicación incorpora una pequeña API REST desarrollada con Express. Actualmente dispone del siguiente endpoint:

Tabla 5. Endpoint de la API REST de la aplicación Makisá.

Método	Ruta	Descripción
POST	/contacto	Guarda una consulta en la base de datos

El flujo de funcionamiento es el siguiente:

- Formulario HTML → Petición POST → Express → MySQL → Respuesta JSON

Ejemplo de datos recibidos (JSON):

```
{
  "nombre": "Usuario",
  "email": "usuario@email.com",
  "mensaje": "Consulta realizada"
}
```

Respuesta en caso de éxito:

```
{ "mensaje": "Consulta guardada correctamente" }
```

Respuesta en caso de error:

```
{ "error": "Error al guardar" }
```

6.9 Despliegue de la aplicación

El frontend se encuentra desplegado mediante la plataforma Netlify. El proceso de publicación se realiza de la siguiente manera:

- Repositorio GitHub → Netlify → <https://makisa.netlify.app>

Cada vez que se realiza un nuevo commit y se envía al repositorio mediante git push, Netlify genera automáticamente una nueva versión de la aplicación. Este proceso de integración continua simplifica enormemente el mantenimiento del proyecto.

6.10 Mantenimiento y ampliaciones futuras

La estructura desarrollada permite incorporar nuevas funcionalidades de forma relativamente sencilla. Entre las posibles ampliaciones se encuentran:

- Incorporación de un panel de administración.
- Gestión dinámica de productos desde la base de datos.
- Sistema de autenticación de usuarios.
- Carrito de compra.
- Pasarela de pagos.
- Sistema de pedidos personalizados.
- Despliegue completo del backend en un servidor online.

- Integración de un sistema de gestión de contenidos.

La separación entre frontend, backend y base de datos facilita la escalabilidad y el mantenimiento de la aplicación en futuras versiones.

7. Pruebas y validación

Durante el proceso de desarrollo se han realizado diversas pruebas para verificar el correcto funcionamiento de cada componente de la aplicación. A continuación se detallan los resultados obtenidos:

Tabla 6. Resultados de las pruebas realizadas sobre la aplicación Makisá.

Prueba realizada	Resultado
Navegación entre páginas	Correcta
Filtrado del catálogo por categorías	Correcto
Carga dinámica de productos	Correcta
Envío del formulario de contacto	Correcto
Almacenamiento de consultas en MySQL	Correcto
Diseño responsive en dispositivos móviles	Correcto
Despliegue en Netlify	Correcto

7.1 Problemas encontrados y soluciones adoptadas

Durante el desarrollo se encontraron los siguientes problemas:

- Diferentes tamaños de imágenes: solucionado mediante las propiedades CSS `aspect-ratio` y `object-fit: contain` en los contenedores de imagen.
- Problemas con rutas relativas: corregidos ajustando la estructura de carpetas y los paths en los archivos HTML y JavaScript.
- Configuración inicial de Netlify: resuelto revisando la documentación oficial y ajustando las opciones de despliegue.
- Integración del backend: solucionado mediante la librería `cors` para permitir peticiones cross-origin desde el frontend desplegado en Netlify al servidor local.

8. Uso de herramientas de Inteligencia Artificial

Durante el desarrollo del proyecto se utilizaron herramientas de Inteligencia Artificial como apoyo al aprendizaje, la resolución de problemas técnicos y la mejora de determinadas funcionalidades de la aplicación. En concreto, se empleó ChatGPT (OpenAI) como herramienta de asistencia durante diferentes fases del desarrollo.

8.1 Diseño de la identidad visual

Prompt utilizado:

"La página tiene actualmente colores azul, verde y magenta, pero el resultado se parece a una página institucional. ¿Qué paleta de colores sería más adecuada para una marca artesanal inspirada en la naturaleza?"

Resultado obtenido: se propuso una nueva paleta basada en tonos vino (#7A004A), magenta (#D80A8C), salmón (#F28A7A), melocotón (#F0C06E) y crema (#FAF7F2). La propuesta fue posteriormente adaptada y ajustada manualmente hasta obtener la identidad visual definitiva de la aplicación.

8.2 Implementación del sistema de filtrado

Prompt utilizado:

"Necesito filtrar las tarjetas del catálogo mediante botones de categoría sin recargar la página."

Resultado obtenido: se obtuvo una propuesta basada en `querySelectorAll()`, `addEventListener()` y atributos `data-filtro` y `data-categoria`. La solución fue modificada posteriormente para adaptarse a la estructura concreta del proyecto y solucionar problemas de distribución de las tarjetas.

8.3 Carga dinámica de productos

Prompt utilizado:

"¿Cómo puedo mostrar la información de un producto utilizando un único archivo HTML y obteniendo el identificador desde la URL?"

Resultado obtenido: se utilizó `URLSearchParams` para leer el parámetro de la URL. Esta solución permitió evitar la creación de una página independiente para cada producto y facilitó la escalabilidad del catálogo.

8.4 Corrección de errores CSS

Prompts utilizados para resolver problemas como la posición de ilustraciones en el hero, la distribución de tarjetas tras aplicar filtros y la deformación de imágenes de producto. La herramienta permitió identificar conflictos entre `display:flex` y `display:grid`, problemas de `object-fit`, duplicidad de reglas CSS, errores en `background-position` y conflictos en media queries.

8.5 Implementación del formulario y backend

Prompt utilizado:

"Tengo conocimientos de MySQL y quiero almacenar las consultas de los usuarios en una base de datos propia. ¿Cómo puedo crear un backend que reciba los datos del formulario y los guarde en MySQL?"

Resultado obtenido: la herramienta proporcionó una guía inicial para crear un servidor con Node.js y Express, configurar el middleware necesario para procesar peticiones JSON, establecer la conexión con MySQL mediante la librería `mysql2`, implementar un endpoint `POST /contacto` y devolver respuestas al cliente. A partir de esta propuesta inicial, se desarrolló y adaptó manualmente el archivo `server.js`, integrándolo con el formulario de contacto de la aplicación y realizando las pruebas necesarias.

9. Conclusiones

El desarrollo de Makisá ha permitido aplicar gran parte de los conocimientos adquiridos durante el ciclo formativo, especialmente en el desarrollo de interfaces web mediante HTML, CSS y JavaScript. Además, el proyecto ha supuesto una oportunidad para ampliar conocimientos en áreas no abordadas en profundidad durante el curso, como el desarrollo backend con Node.js y Express, la creación de APIs REST y la comunicación con bases de datos MySQL.

Desde el punto de vista técnico, se ha conseguido desarrollar una aplicación web completa, funcional y adaptable a distintos dispositivos, cumpliendo los objetivos planteados inicialmente. Asimismo, el proyecto ha permitido comprender la importancia del control de versiones mediante Git y GitHub, así como los procesos de despliegue de aplicaciones web utilizando plataformas como Netlify.

Finalmente, la realización de este trabajo ha servido para adquirir una visión más global del desarrollo de aplicaciones web, integrando frontend, backend y bases de datos en un único proyecto cohesionado.

9.1 Líneas de trabajo futuras

Aunque la aplicación cumple los objetivos planteados inicialmente, se han identificado diversas funcionalidades que podrían implementarse en futuras versiones:

- Panel de administración para gestionar productos.
- Sistema de autenticación de usuarios.
- Carrito de compra y pasarela de pagos.
- Gestión de stock.
- Sistema de búsqueda avanzada.
- Integración con redes sociales.
- Sistema de pedidos personalizados.
- Subida dinámica de productos desde la base de datos.
- Despliegue completo del backend en un servidor online.
- Implementación de medidas adicionales de seguridad y validación.

10. Bibliografía y referencias

Las siguientes referencias han sido utilizadas en el desarrollo del proyecto, citadas en formato APA 7:

Referencias generales

Bootstrap. (s. f.). Bootstrap documentation. <https://getbootstrap.com/>

Crockford, D. (2008). JavaScript: The good parts. O'Reilly Media.

Express.js. (s. f.). Express - Node.js web application framework. <https://expressjs.com/>

Flanagan, D. (2020). JavaScript: The definitive guide (7.^a ed.). O'Reilly Media.

Font Awesome. (s. f.). Font Awesome documentation. <https://fontawesome.com/>

Freeman, E., & Robson, E. (2014). Head first HTML and CSS (2.^a ed.). O'Reilly Media.

Git. (s. f.). Git documentation. <https://git-scm.com/doc>

GitHub. (s. f.). GitHub documentation. <https://docs.github.com/>

Mozilla Developer Network. (s. f.). CSS: Cascading Style Sheets. <https://developer.mozilla.org/es/docs/Web/CSS>

Mozilla Developer Network. (s. f.). Fetch API. https://developer.mozilla.org/es/docs/Web/API/Fetch_API

Mozilla Developer Network. (s. f.). HTML: HyperText Markup Language. <https://developer.mozilla.org/es/docs/Web/HTML>

Mozilla Developer Network. (s. f.). JavaScript. <https://developer.mozilla.org/es/docs/Web/JavaScript>

Mozilla Developer Network. (s. f.). URLSearchParams. <https://developer.mozilla.org/es/docs/Web/API/URLSearchParams>

Mozilla Developer Network. (s. f.). Using data attributes. https://developer.mozilla.org/en-US/docs/Learn/HTML/Howto/Use_data_attributes

Mozilla Developer Network. (s. f.). Using the Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MySQL. (s. f.). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/>

MySQL. (s. f.). mysql2 documentation. <https://www.npmjs.com/package/mysql2>

Netlify. (s. f.). Netlify documentation. <https://docs.netlify.com/>

Node.js. (s. f.). Node.js documentation. <https://nodejs.org/en/docs/>

Resig, J., & Bibeault, B. (2016). Secrets of the JavaScript ninja (2.^a ed.). Manning Publications.

W3C. (2021). HTML living standard. <https://html.spec.whatwg.org/>

W3C. (2021). CSS snapshot 2021. <https://www.w3.org/TR/css-2021/>

Diseño responsive y experiencia de usuario

Krug, S. (2014). Don't make me think, revisited: A common sense approach to web usability (3.^a ed.). New Riders.

Marcotte, E. (2014). Responsive web design (2.^a ed.). A Book Apart.

APIs y desarrollo web

Richardson, L., & Ruby, S. (2007). RESTful web services. O'Reilly Media.

Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. IEEE Internet Computing, 14(6), 80–83. <https://doi.org/10.1109/MIC.2010.145>

Recursos gráficos

Font Awesome. (s. f.). Icons. <https://fontawesome.com/icons>

Google Fonts. (s. f.). <https://fonts.google.com/specimen/Cormorant> Cormorant Garamond.
<https://fonts.google.com/specimen/Cormorant+Garamond>

Google Fonts. (s. f.). Poppins. <https://fonts.google.com/specimen/Poppins>